Jenkins Basic

VNTECHIES

Today's Lesson

CI/CD Concepts - Understanding continuous integration and deployment

Jenkins Architecture - Master, agents, and components

Job Types - Freestyle, Pipeline, Multi-branch

Writing Jenkinsfiles

VNTECHIES

CI/CD Concepts

VNTECHIES

CI/CD Concepts

The Problem: Manual Deployment

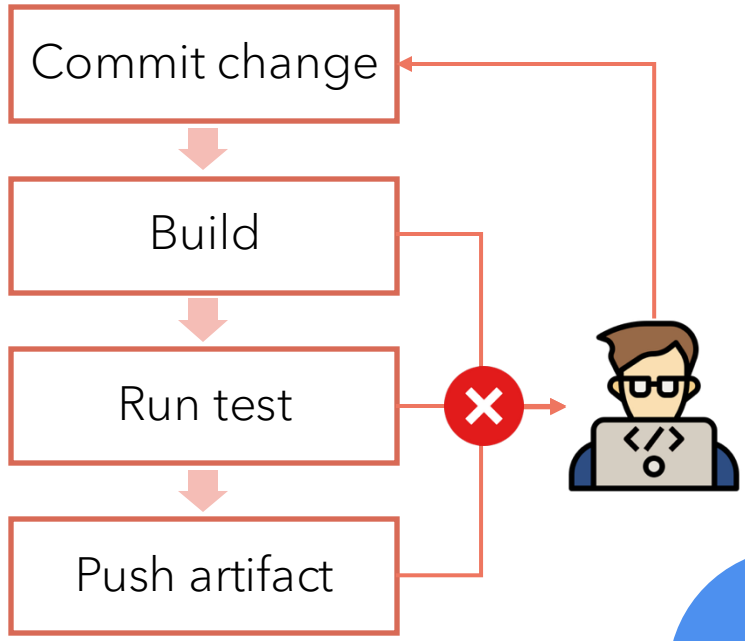
- 😰 Manual builds are error-prone
- ⏰ Time-consuming process
- 🐛 Late bug detection
- 📉 Inconsistent environments
- 👥 Knowledge silos
- 🔥 "Works on my machine"

The Solution: CI/CD

- ✅ Automated builds and tests
- ⚡ Fast feedback loops
- 🔍 Early bug detection
- 🎯 Consistent deployments
- 🤝 Team collaboration
- 🚀 Faster time to market

VNTECHIES

Continuous Integration (CI)



Continuous Integration (CI) is the frequent, automatic integration of code. All new and modified code is automatically tested with the master code.

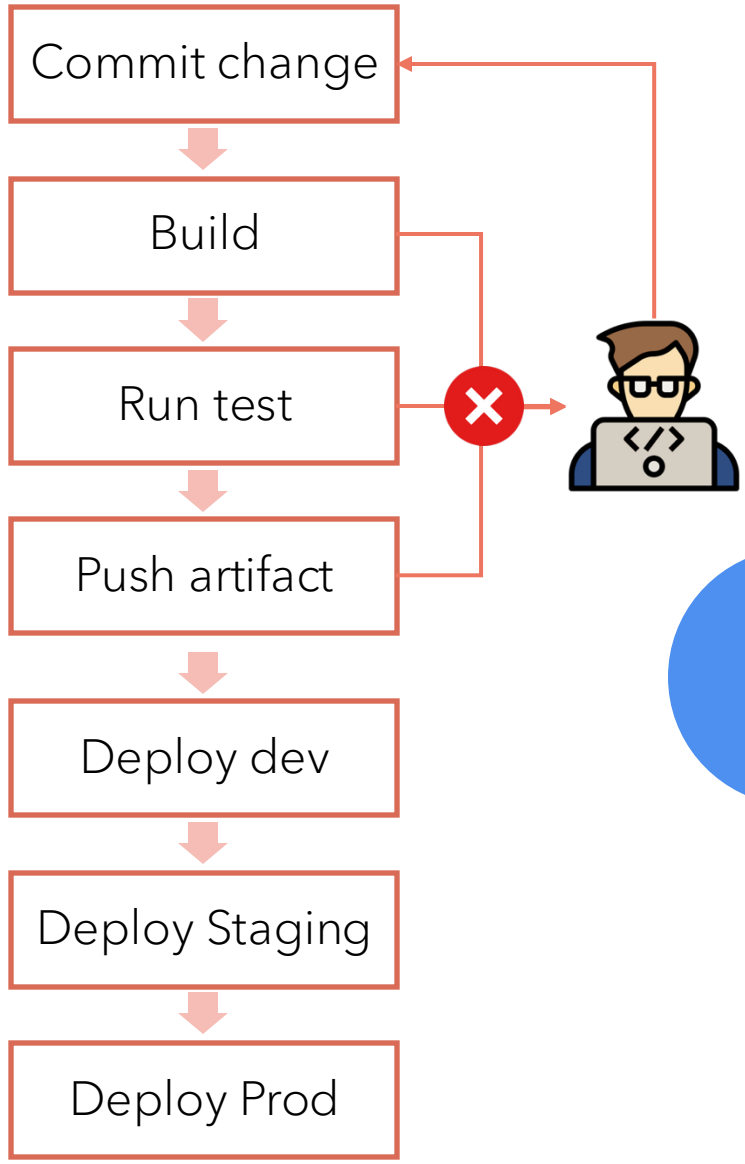
Key Principles

- Commit code frequently (daily)
- Automated build process
- Automated testing
- Fast feedback to developers
- Fix broken builds immediately

VNTECHIES

Continuous Delivery (CD)

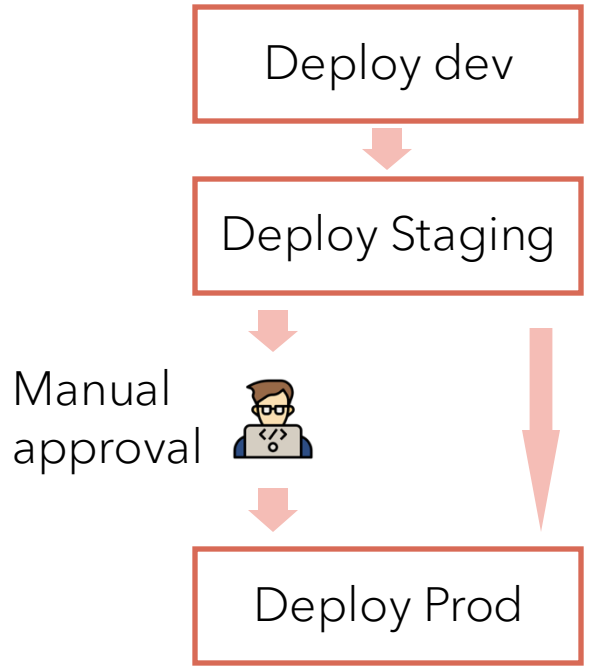
Continuous Delivery (CD) is the natural extension of CI. It ensures that the code is always ready to be deployed, although manual approval is required to actually deploy the software to production.



VNTECHIES

Continuous Deployment

Continuous Deployment (CD) automatically deploys all validated changes to production. Frequent feedback enables issues to be found and fixed quickly.



VNTECHIES

Continuous Delivery vs Continuous Deployment

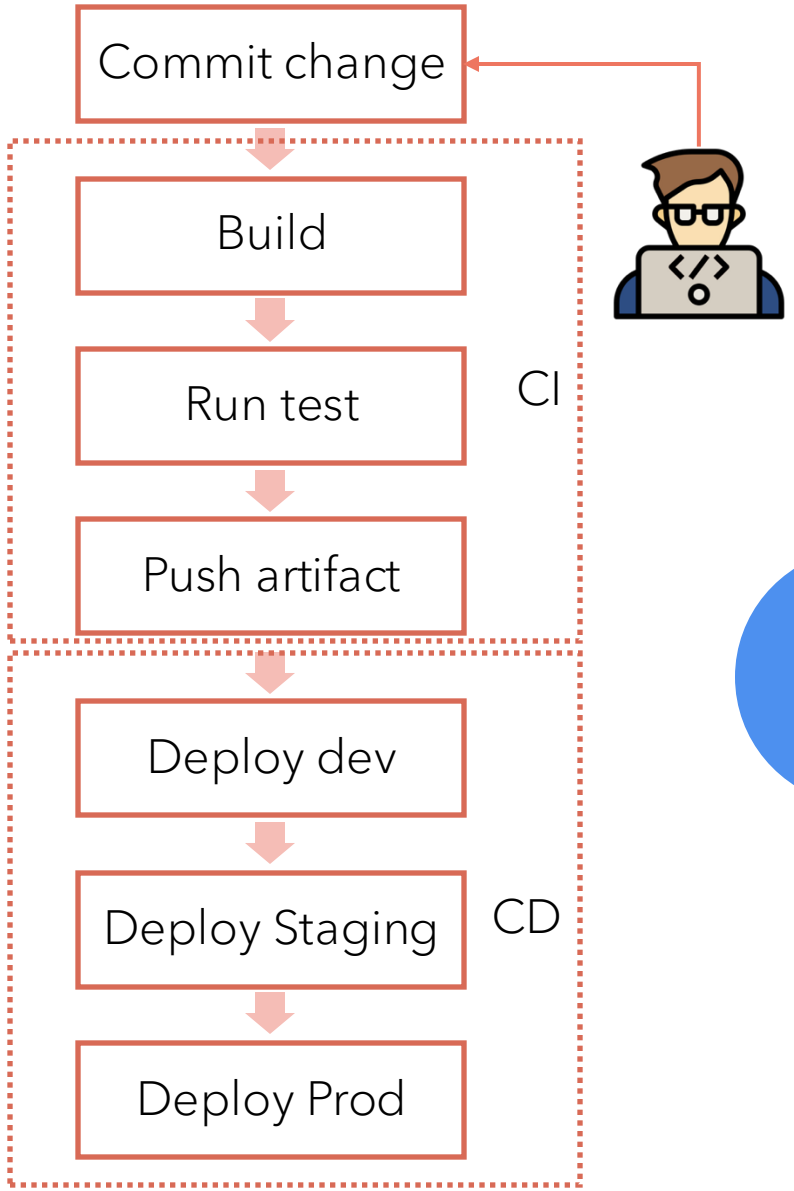
- Automated deployment to **staging**
- Manual approval for production
- Always in deployable state
- Reduces deployment risk
- Business decides when to release

- Automated deployment to **production**
- No manual intervention
- Every change goes live automatically
- Requires high confidence in tests
- Fastest time to market

VNTECHIES

CI/CD Pipeline Stages

- **Source** - Code repository (Git)
- **Build** - Compile and package application
- **Test** - Run automated tests (unit, integration, security)
- **Deploy** - Release to environments (dev, staging, production)
- **Monitor** - Track performance and errors



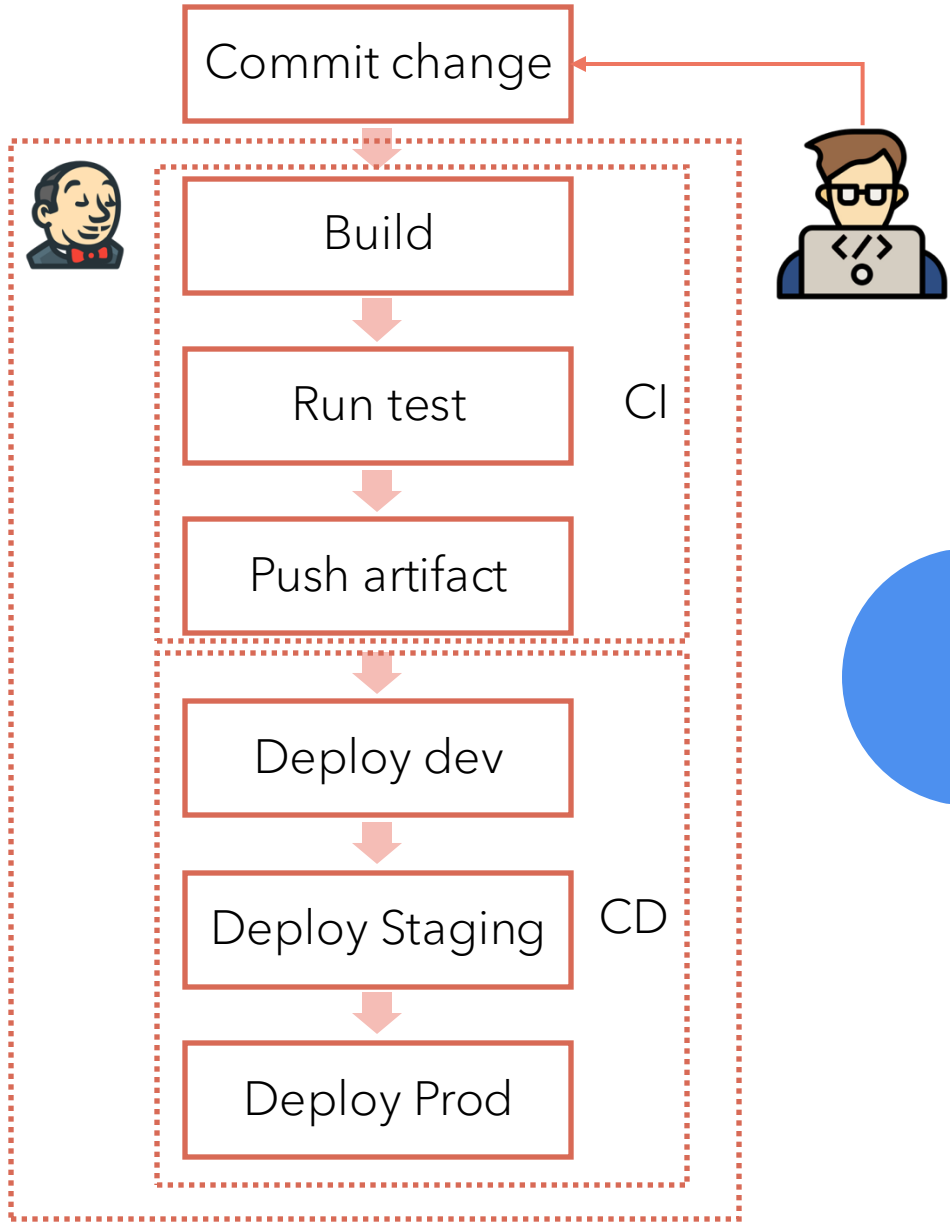
VNTECHIES

Jenkins Fundamental

VNTECHIES

What is Jenkins?

Jenkins is an **open-source automation server** written in Java. It is widely used to **automate the process of building, testing, and deploying applications**. Jenkins supports **Continuous Integration (CI)** and **Continuous Delivery (CD)**, which help development teams integrate code changes frequently, verify those changes automatically, and deliver updates faster and more reliably.



VNTECHIES

Jenkins Architecture

The Jenkins architecture follows a **master-agent (controller-agent)** model:

1. Jenkins Controller (Master)

- Manages the overall Jenkins environment.
- Handles scheduling of jobs and dispatching build requests to agents.
- Monitors the agents and records build results.
- Provides the web interface for users to interact with Jenkins.

2. Jenkins Agents (Slaves/Nodes)

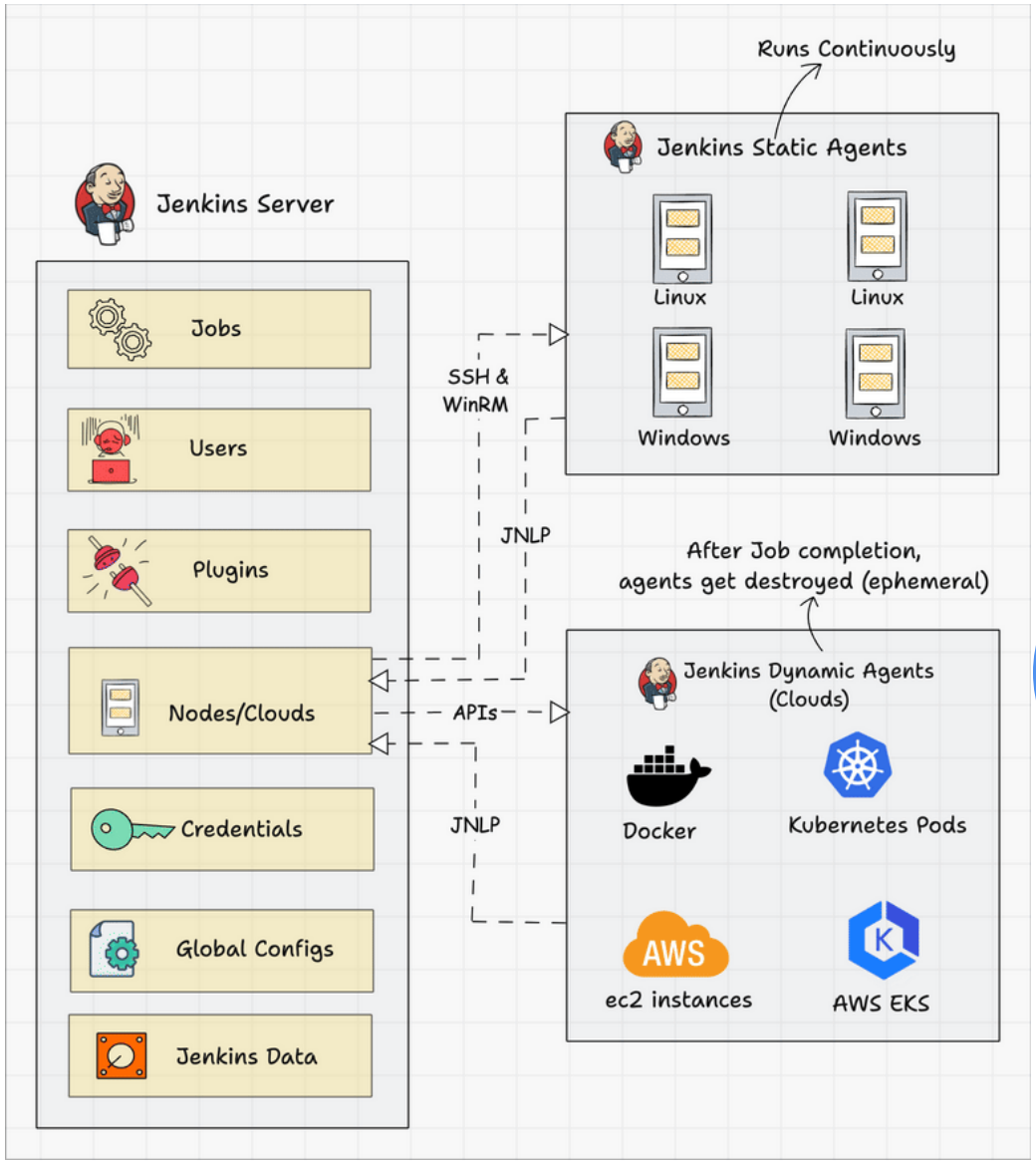
- Execute build jobs as assigned by the controller.
- Can run on different platforms (Linux, Windows, macOS, etc.).
- Help scale Jenkins horizontally by distributing workloads.

3. Build Jobs / Pipelines

- Define what to build, test, or deploy, and how to do it.
- Can be configured via UI or as **Jenkinsfile** (Pipeline as Code).

4. Plugins

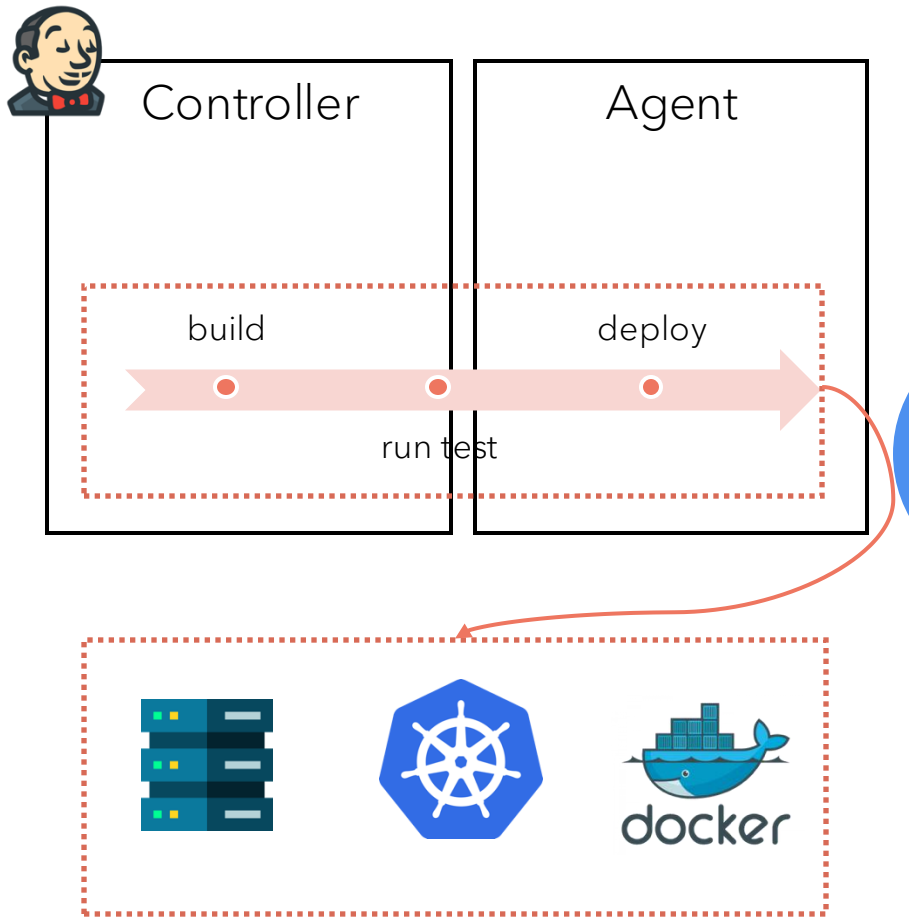
- Jenkins' biggest strength – over 1,800 plugins enhance Jenkins capabilities, integrating with tools such as Git, Maven, Docker, Kubernetes, and more.



VNTECHIES

What Jenkins can do?

- In Jenkins, we define jobs (pipeline) to do what we need:
- Example:
  - Deploy nodejs app to server (could be VN or docker container)
  - Deploy to K8s a
  - Run adhoc job (install dependenies, backup job)
  - Or any automation job
- Basicly, Jenkins can do all the thing that supported CLI, RestAPI (which most of tech stack already supported)



VNTECHIES

Jenkins Pipeline

- A **Jenkins Pipeline** is a **suite of plugins** that supports implementing and integrating **Continuous Integration (CI)** and **Continuous Delivery (CD)** workflows in Jenkins using **code**.

- In simple terms:
  A Jenkins Pipeline defines the **process of building, testing, and deploying** your application in an **automated, repeatable, and version-controlled** manner – described in a text file usually called a **Jenkinsfile**.

```
pipeline {
    agent any // Specifies that the pipeline can run on any
available agent

    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

VNTECHIES

Jenkins Pipeline

- Each Jenkins Job (pipleline) can be defined as code
- Pipeline can be trigger by:
  - Manually (user click)
  - Auto Trigger (when PR created, code commit)
  - Scheduled by cronjob

```
pipeline {
    agent any // Specifies that the pipeline can run on any
available agent

    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

VNTECHIES

Jenkins Pipeline

Pipeline:

- A **Pipeline** describes the *overall workflow*.
- A **Stage** separates the pipeline into *logical sections*.
- A **Step** executes *individual actions* within a stage.

Concept	Represents	Example
Pipeline	Complete CI/CD process (end-to-end automation)	Entire Jenkinsfile
Stage	Logical phase in the pipeline (build/test/deploy)	stage('Test') { ... }
Step	Single task inside a stage	sh 'mvn test', echo 'Done'

```
pipeline {
    agent any // Specifies that the pipeline can run on any
available agent

    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

VNTECHIES

Declarative Pipeline VS Scripted Pipeline

Decalrative

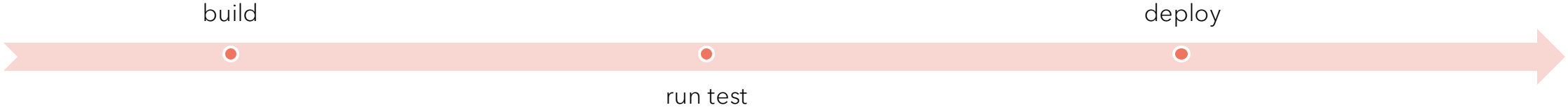
```
pipeline {
    agent any // Specifies that the pipeline can run on any
available agent

    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

Scripted

```
node {
    stage('Build') {
        echo 'Building...'
    }
    stage('Test') {
        echo 'Running tests...'
    }
    stage('Deploy') {
        echo 'Deploying...'
    }
}
```

VNTECHIES

Declarative Pipeline VS Scripted Pipeline



Decalrative VS Scripted

Feature	Declarative Pipeline	Scripted Pipeline
Ease of Use	Easier, structured syntax	Flexible, but more complex
Syntax	pipeline { stages { ... } }	node { stage('...') { ... } }
Error Handling	Built-in post blocks	Manual try-catch handling
Flexibility	Limited to defined structure	Fully customizable (Groovy code)
Recommended For	Most CI/CD scenarios	Advanced, dynamic logic

Since scripted pipeline provided more feature and flexible, It recommended for CICD pipeline

VNTECHIES

Jenkins Plugin

A **Jenkins Plugin** is an **extension or add-on** that adds extra features or integrations to Jenkins.

- Jenkins by itself provides only basic automation functionality.
- Plugins allow Jenkins to **integrate with tools** like Git, Maven, Docker, Kubernetes, Slack, etc., or to add **new pipeline steps, UI features, or authentication methods**.

Example:

- *Git Plugin* → Integrates Jenkins with Git.
- *Pipeline Plugin* → Enables Jenkins pipelines.
- *Slack Notification Plugin* → Sends build results to Slack.
- *Docker Plugin* → Builds and manages Docker containers.
- *SonarQube Plugin* → Integrates code quality analysis.

VNTECHIES

Jenkins Plugin

Jenkins Plugin can be install via UI.

And can be used in Pipeline Code

```
node {
    try {
        stage('Checkout Source') {
            echo 'Checking out source code...'
            git branch: 'main', url: 'https://github.com/example/repo.git'
        }

        stage('Build') {
            echo 'Building the application...'
        }

        stage('Post-Build Notification') {
            echo 'Build completed successfully.'
            slackSend(channel: '#ci-cd', message: "✅ Build Success:
${env.JOB_NAME}")
        }

    } catch (err) {
        echo "Build failed: ${err}"
        slackSend(channel: '#ci-cd', message: "❌ Build Failed:
${env.JOB_NAME}")
        currentBuild.result = 'FAILURE'
        throw err
    } finally {
        echo 'Pipeline execution finished.'
    }
}
```

VNTECHIES

Practice Lab